Design and Analysis of Algorithms

- ***Course Code***: BCSE204L
- ***Course Type***: Theory (ETH)
- ***Slot***: A1+TA1 & & A2+TA2
- ***Class ID***: VL2023240500901
  VL2023240500902

A1+TA1

Day	Start	End
Monday	08:00	08:50
Wednesday	09:00	09:50
Friday	10:00	10:50

A2+TA2

Day	Start	End
Monday	14:00	14:50
Wednesday	15:00	15:50
Friday	16:00	16:50

Syllabus- Module 6

Module:6	Randomized algorithms	5 hours

Randomized quick sort - The hiring problem - Finding the global Minimum Cut

Hiring problem

- The hiring problem is a classic problem in randomized algorithms where you need to find a "good enough" candidate from a pool of applicants while minimizing the number of interviews conducted.

- The **solution** to the hiring problem typically (brute force approach) involves interviewing candidates one by one and making a decision on whether to hire or reject each candidate immediately after the interview.
- The **input** is a **list of candidates**, and the **output** is the candidate that is chosen for hire.
- The Hiring Problem is related to Las Vegas algorithms. Las Vegas algorithms always produce the correct answer but their running time is a random variable whose expectation is bounded.

The deterministic approach (interviewing every candidate) has a time complexity of **O(n)** for comparisons. However, it doesn't guarantee finding the "best" candidate efficiently.

Hiring problem

How does the randomized algorithm solve the limitation of the hiring problem?

- The randomized hiring algorithm achieves a balance between interview efficiency and candidate quality. By randomly interviewing candidates, it avoids the worst-case scenario of interviewing everyone (deterministic approach) and potentially misses the best candidate early on.
- Its expected time complexity is O(n/e), where e is Euler's number (approximately 2.71828).
- This expected time complexity arises from the fact that each candidate has a 1/e probability of being the best candidate. So, on average, we need to interview approximately n/e candidates before finding the best one.

Hiring problem

Algorithm

1. Let best_seen be None (initially no best candidate)
2. For each candidate i in random order (1 to n):
   - If candidate[i] is better than best_seen:
     - Set best_seen to candidate[i]
     - Hire candidate[i] (stop interviewing)
3. If no candidate was hired (loop finishes), hire best_seen (might not be the absolute best)

Sample Input: Consider 5 candidates (A, B, C, D, E) with the following qualifications (higher number is better):

A: 3
B: 7
C: 1
D: 5
E: 2

Sample Output (one possibility):
Imagine the random order is B, D, E, A, C. The algorithm would:
Interview B (better than none, becomes best_seen)
Interview D (better than B, becomes best_seen)
Interview E (not better than D, discarded)
Interview A (better than E, becomes best_seen)
Interview C (not better than A, discarded)

Syllabus- Module 6

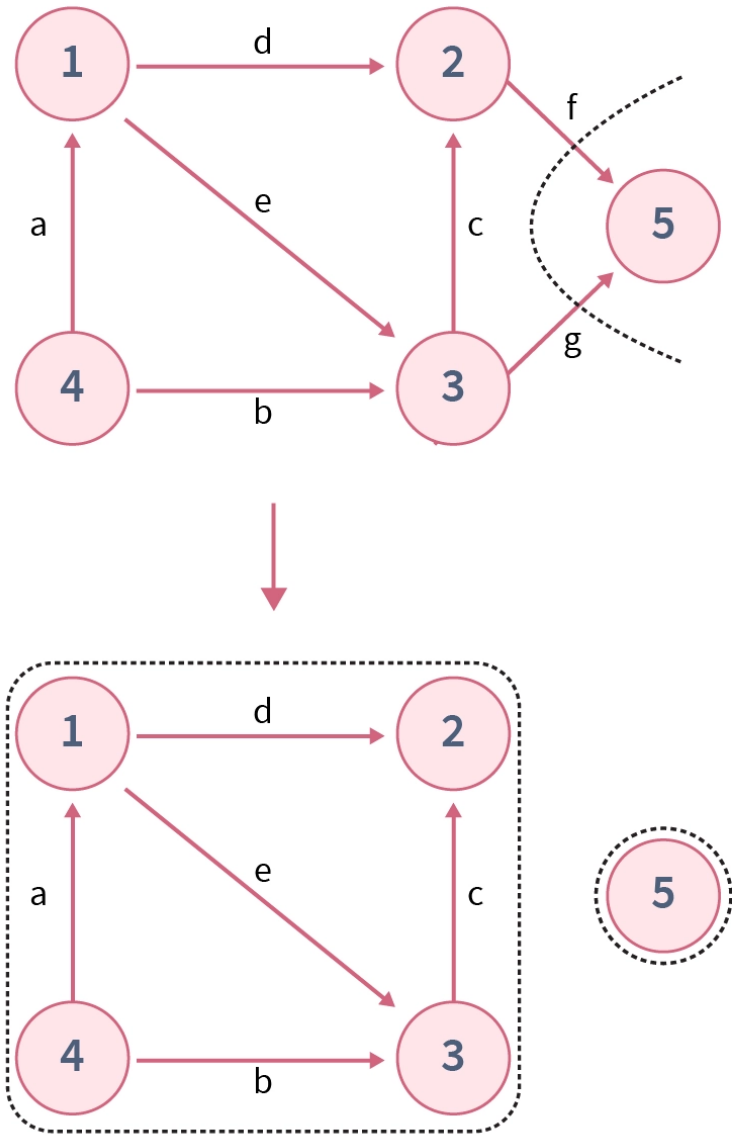
Module:6	Randomized algorithms	5 hours

Randomized quick sort - The hiring problem - Finding the **global Minimum Cut**

Finding the **global Minimum Cut**

In graph theory, the Global Minimum Cut problem seeks to find the **smallest set of edges** whose removal separates the graph into two disconnected (non-overlapping) parts. These edges represent the weakest connection points within the graph.

Finding the **global Minimum Cut**



- **Input**:
  - Vertices: A, B, C, D
  - Edges: (A-B), (A-C), (B-C), (C-D)
- **Output**:
  - Cut: (C-D)

Finding the **global Minimum Cut**

Input: A connected, undirected graph G = (V, E) with positive edge capacities (weights) cap(u, v), representing the "strength" of the connection between vertices u and v.

Output: A set of edges S that constitutes a minimum cut. This means removing the edges in S disconnects the graph into two parts, and the sum of the capacities of the edges in S is the minimum compared to all other possible cuts in the graph.

Finding the **global Minimum Cut**

Sample Input: Vertices: A, B, C, D, E
Edges: (A-B: 3), (A-C: 2), (B-C: 1), (B-D: 7), (C-D: 4), (C-E: 5), (D-E: 6)

Sample Output (one possibility): The minimum cut separating vertices {A,C} and {B, C} is {C, D}, with a total capacity of 7.

Deterministic Algorithms: Brute-force approaches trying all possible cuts have a time complexity of O(V^E), which is impractical for large graphs.
Karger's Randomized Algorithm: This efficient algorithm achieves a time complexity of O(n^2 log n), where n is the number of vertices in the graph.

Any Question?

Thank You!

FOR YOUR ATTENTION